Vulnerability Assessment Report

Generated on 2026-03-15 19:18 UTC

Total vulnerabilities	8
Critical	0
High	0
Medium	3
Low	3
Info	2

Executive Summary

This report lists unique findings from the security scan, grouped by severity. Address critical and high severity issues first.

Severity	Count	Description
Critical	0	Immediate action required.
High	0	Fix as soon as possible.
Medium	3	Plan remediation.
Low	3	Consider in next release.
Info	2	Informational only.

Findings

1. Session Management Response Identified

INFORMATIONAL

CWE: -1

Description

The given response has been identified as containing a session management token. The 'Other Info' field contains a set of header tokens that can be used in the Header Based Session Management Method. If the request is in a context which has a Session Management Method set to "Auto-Detect" then this rule will change the session management to use the tokens identified.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

This is an informational alert rather than a vulnerability and so there is nothing to fix.

2. Cookie No HttpOnly Flag

LOW

CWE: 1004

Description

A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

Ensure that the HttpOnly flag is set for all cookies.

Tags

CWE-1004

OWASP_2021_A05

WSTG-v42-SESS-02

SYSTEMIC

OWASP_2017_A06

3. Cookie without SameSite Attribute

LOW

CWE: 1275

Description

A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

Tags

OWASP_2021_A01

WSTG-v42-SESS-02

SYSTEMIC

CWE-1275

OWASP_2017_A05

4. Server Leaks Version Information via "Server" HTTP Response Header Field

LOW

CWE: 497

Description

The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details.

Tags

OWASP_2021_A05

SYSTEMIC

OWASP_2017_A06

WSTG-v42-INFO-02

CWE-497

5. Missing Anti-clickjacking Header

MEDIUM

CWE: 1021

Description

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Tags

OWASP_2021_A05

CWE-1021

SYSTEMIC

WSTG-v42-CLNT-09

OWASP_2017_A06

6. Content Security Policy (CSP) Header Not Set

MEDIUM

CWE: 693

Description

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.

Tags

OWASP_2021_A05

SYSTEMIC

CWE-693

OWASP_2017_A06

7. Absence of Anti-CSRF Tokens

MEDIUM

CWE: 352

Description

No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable URL/form actions in a repeatable way. The nature of the attack is that CSRF exploits the trust that a web site has for a user. By contrast, cross-site scripting (XSS) exploits the trust that a user has for a web site. Like XSS, CSRF attacks are not necessarily cross-site, but they can be. Cross-site request forgery is also known as CSRF, XSRF, one-click attack, session riding, confused deputy, and sea surf. CSRF attacks are effective in a number of situations, including: * The victim has an active session on the target site. * The victim is authenticated via HTTP auth on the target site. * The victim is on the same local network as the target site. CSRF has primarily been used to perform an action against a target site using the victim's privileges, but recent techniques have been discovered to disclose information by gaining access to the response. The risk of information disclosure is dramatically increased when the target site is vulnerable to XSS, because XSS can be used as a platform for CSRF, allowing the attack to operate within the bounds of

the same-origin policy.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Recommendation

Phase: Architecture and Design Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid. For example, use anti-CSRF packages such as the OWASP CSRFGuard. Phase: Implementation Ensure that your application is free of cross-site scripting issues, because most CSRF defenses can be bypassed using attacker-controlled script. Phase: Architecture and Design Generate a unique nonce for each form, place the nonce into the form, and verify the nonce upon receipt of the form. Be sure that the nonce is not predictable (CWE-330). Note that this can be bypassed using XSS. Identify especially dangerous operations. When the user performs a dangerous operation, send a separate confirmation request to ensure that the user intended to perform that operation. Note that this can be bypassed using XSS. Use the ESAPI Session Management control. This control includes a component for CSRF. Do not use the GET method for any request that triggers a state change. Phase: Implementation Check the HTTP Referer header to see if the request originated from an expected page. This could break legitimate functionality, because users or proxies may have disabled sending the Referer for privacy reasons.

Tags

OWASP_2021_A01

SYSTEMIC

WSTG-v42-SESS-05

OWASP_2017_A05

CWE-352

8. User Agent Fuzzer

INFORMATIONAL

CWE: 0

Description

Check for differences in response based on fuzzed User Agent (eg. mobile sites, access as a Search Engine Crawler). Compares the response statuscode and the hashcode of the response body with the original response.

URL	http://192.168.56.101/dvwa/vulnerabilities/xss_r/
Method	GET

Tags

SYSTEMIC